《区块链基础及应用》Ex2实验报告

学院：计算机及网络空间安全学院　　专业：计算机科学与技术　　姓名：李梦涛　　学号：2412483

Ex2

一、项目构成

github地址：KaGaMi9999/nku-2026-blockchains

Blockchains

- Lab
  - Ex2
    - codes
      - config.py：配置我的私钥、地址与老师收币地址（沿用 Ex0/Ex1）
      - ex1.py：Ex1 的 P2PKH 实现，Ex2a 复用其中的 `send_from_P2PKH_transaction`
      - ex2a.py：完成多签锁仓脚本 `ex2a_redeemScript` / `ex2a_txout_scriptPubKey`，并广播锁仓交易
      - ex2b.py：完成多签解锁脚本 `multisig_scriptSig`，并广播赎回交易
      - keygen.py：生成客户私钥和地址
      - requirements.txt：环境要求（python-bitcoinlib、requests）
      - split_test_coins .py：Ex1 中的分币脚本（Ex2 直接复用其分币结果）
      - utils.py：封装了构造交易、签名、广播的工具函数
    - documents
      - Ex2.pdf：老师下发的实验说明
      - ex2a.py：老师下发的启动代码
      - ex2b.py：老师下发的启动代码
    - MyReports
      - BlockChain-Ex2-2412483.md：撰写的实验报告
      - 生成的三个私钥和地址.jpg：运行 keygen.py 生成三个客户密钥的结果
      - Ex2a.jpg：运行 ex2a.py 广播锁仓交易的记录
      - Ex2b.png：运行 ex2b.py 广播赎回交易的记录

二、实验内容

实验目标

本实验要求完成一笔涉及**四方**的多签名（multisig）交易：第一方是**银行**（这里假定自己就是银行），另外三方是**客户**。这笔钱必须由「银行 + 三个客户中的任意一个」**共同签名**才能赎回（即 2-of-4 多签），不能只靠银行或只靠客户单方。实验分两步：

1. （a）生成多签锁仓交易，把币锁进多签地址；
2. （b）构造解锁脚本赎回这笔钱，要求锁定脚本（scriptPubKey）尽可能小，且任意合法签名组合都能赎回至老师的 faucet 地址。

步骤 1：生成客户密钥

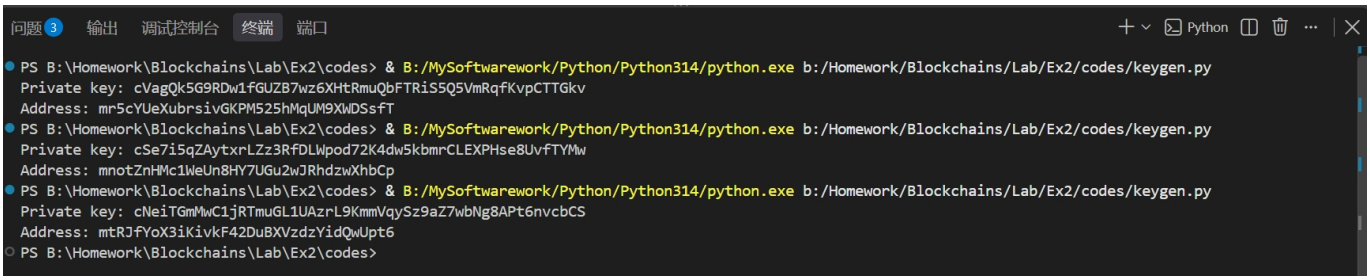
我们扮演银行角色，银行的公钥/私钥直接用 `config.py` 里我自己的密钥（`my_public_key` / `my_private_key`）。三个客户的密钥则用 `keygen.py` 生成，生成后填入 `ex2a.py`：

```
cust1_private_key = CBitcoinSecret(
    'cVagQk5G9RDw1fGUZB7wz6XHtRmuQbFTRiS5Q5VmRqfKvpCTTGkv')
cust1_public_key = cust1_private_key.pub

cust2_private_key = CBitcoinSecret(
    'cSe7i5qZAytxrLZz3RfDLWpod72K4dw5kbmrCLEXPHse8UvfTYMw')
cust2_public_key = cust2_private_key.pub

cust3_private_key = CBitcoinSecret(
    'cNeiTGmMwC1jRTmuGL1UAzrL9KmmVqySz9aZ7wbNg8APt6nvcbCS')
cust3_public_key = cust3_private_key.pub
```

其中私钥开头的 `c` 是 testnet 的压缩公钥编码前缀，`custX_private_key.pub` 会推导出对应的 33 字节压缩公钥。



步骤 2：构造多签锁仓脚本（ex2a.py）

「银行 + 任意一个客户」共同签名，等价于 4 个公钥里需要 2 个有效签名，因此用 `OP_CHECKMULTISIG` 表示成 2-of-4：

```
ex2a_redeemScript = CScript([
    OP_2,              # 需要 2 个有效签名（银行 + 任意一个客户）
    my_public_key,     # 银行公钥（即自己的公钥）
    cust1_public_key,  # 客户1公钥
    cust2_public_key,  # 客户2公钥
    cust3_public_key,  # 客户3公钥
    OP_4,              # 共有 4 个公钥
    OP_CHECKMULTISIG,  # 校验多签名
])
```

`OP_CHECKMULTISIG` 的执行规则是：先弹出 `m`（此处为 2）个签名，再弹出 `n`（此处为 4）个公钥，然后逐个校验，需要至少有 m 个签名能对应上前面的公钥。注意脚本里公钥的顺序决定了后面解锁脚本里签名的顺序，所以我们统一约定顺序为 **银行 -> 客户1 -> 客户2 -> 客户3。**

这里有一个关键的**标准性**问题：比特币核心节点的 `IsStandard` 策略规定，**裸多签（bare multisig）最多只允许 3 个公钥（x-of-3）**，超过 3 个公钥的裸多签会被当作非标准交易拒收（报错 `-26: scriptpubkey`）。而我们的 2-of-4 有 4 个公钥，直接作为锁定脚本是无法广播的。

解决办法是使用 **P2SH（Pay-to-Script-Hash）**：把上面这段多签脚本哈希后，锁定脚本只保留一个固定 23 字节的 `OP_HASH160 <hash160(redeemScript)> OP_EQUAL`，真正的多签逻辑藏进 redeem script，等到花费时才披露：

```
ex2a_txout_scriptPubKey = ex2a_redeemScript.to_p2sh_scriptPubKey()
```

这样锁定脚本「尽可能小」（23 字节），又满足标准性要求。`to_p2sh_scriptPubKey()` 内部就是计算 `Hash160(redeemScript)` 并拼成 `OP_HASH160 <20字节哈希> OP_EQUAL`。

构造出来的脚本（十六进制）如下：

- redeem script（139 字节）：`52 21 <33字节银行公钥> 21 <33字节客户1公钥> 21 <33字节客户2公钥> 21 <33字节客户3公钥> 54 ae`，其中 `52` =OP_2、`21` =压入33字节、`54` =OP_4、`ae` =OP_CHECKMULTISIG；
- P2SH 锁定脚本（23 字节）：`a9140ae215aed5d954cc3c8178826aaeed5529dbd53687`，对应多签地址 `2MtEmaqpYVu7BgcjjEmCVUB4jZdDxWk72pf`。

步骤 3：广播锁仓交易（ex2a.py）

`ex2a.py` 复用 Ex1 的 `send_from_P2PKH_transaction`，花费分币交易里的一个输出，把币锁进上面的 P2SH 多签地址。main 函数参数如下：

```
if __name__ == '__main__':
    amount_to_send = 0.00009          # 实发 9000 sat
    txid_to_spend = (
        'a94c1641cd0311ed2b6e33599829a128f597774e71dedb71bfe2e1699878ccb9')
    utxo_index = 1                    # 分币交易的 1 号输出（0 号已在 Ex1 用过）

    response = send_from_P2PKH_transaction(
        amount_to_send, txid_to_spend, utxo_index,
        ex2a_txout_scriptPubKey)
    print(response.status_code, response.reason)
    print(response.text)
```

运行后广播成功（HTTP 200），返回的交易哈希为：

```
d3adf29cbff2aaab2d19c2ef2d817d0f1e0e93e771df3fb5ae8057ba9ce55122
```

链上查询可知，这笔交易把 0.00010000 tBTC（10000 聪）作为输入，向 P2SH 多签地址 `2MtEmaqpYVu7BgcjjEmCVUB4jZdDxWk72pf` 的 **0 号输出**写入了 **9000 聪**，差额 1000 聪作为手续费。



步骤 4：构造多签解锁脚本（ex2b.py）

赎回时，`multisig_scriptSig` 需要生成解锁脚本。因为锁仓用的是 P2SH，所以签名要针对 redeem script（而不是 P2SH 锁定脚本）来计算签名哈希，并且最后要把整段 redeem script 压栈：

```
def multisig_scriptSig(txin, txout, txin_scriptPubKey):
    # P2SH 中签名要针对 redeem script 计算签名哈希
    bank_sig = create_OP_CHECKSIG_signature(txin, txout, ex2a_redeemScript,
                                            my_private_key)
    cust1_sig = create_OP_CHECKSIG_signature(txin, txout, ex2a_redeemScript,
                                            cust1_private_key)
    return [OP_0, bank_sig, cust1_sig, ex2a_redeemScript]
```

解锁脚本四个元素的含义：

- `OP_0`：`OP_CHECKMULTISIG` 有一个历史 bug，会多弹出一个堆栈元素，所以脚本开头必须压入一个占位符；
- `bank_sig`：银行签名，对应 redeem script 中第 1 个公钥（银行）；
- `cust1_sig`：客户1签名，对应 redeem script 中第 2 个公钥（客户1）。换成 `cust2_sig` / `cust3_sig` 也同样能赎回，只要签名顺序与公钥顺序对应即可；
- `ex2a_redeemScript`：P2SH 校验时需要用这段脚本算哈希，与锁定脚本里的哈希比对。

步骤 5：广播赎回交易（ex2b.py）

赎回交易花费锁仓交易的那 9000 聪，发回老师的 faucet 地址。main 函数参数如下：

```
if __name__ == '__main__':
    amount_to_send = 0.00008          # 实发 8000 sat
    txid_to_spend = 'd3adf29cbff2aaab2d19c2ef2d817d0f1e0e93e771df3fb5ae8057ba9ce55122'
    utxo_index = 0                    # 锁仓交易的唯一输出

    txin_scriptPubKey = ex2a_txout_scriptPubKey
    txout_scriptPubKey = P2PKH_scriptPubKey(faucet_address)

    response = send_from_multisig_transaction(
        amount_to_send, txid_to_spend, utxo_index,
        txin_scriptPubKey, txout_scriptPubKey)
    print(response.status_code, response.reason)
    print(response.text)
```

运行后广播成功（HTTP 200），返回的交易哈希为：

```
de63746c6fe61cd948e5b9fdf91610ccd1ef559ae2d39ffef9cf6fc8678009c3
```

链上查询可知，这笔交易花掉了锁仓交易的 0 号输出（9000 聪），解锁脚本中披露了 redeem script `OP_2 <4公钥> OP_4 OP_CHECKMULTISIG`，向老师地址 `mv4rnyY3Su5gjcDNzbMLKBQkBicCtHUtFB` 发送了 **8000 聪**，差额 1000 聪作为手续费。至此完成了「银行 + 客户1」两个签名共同赎回的全过程。

